

기술 스택 및 선택 근거

프로젝트명 : SenSa

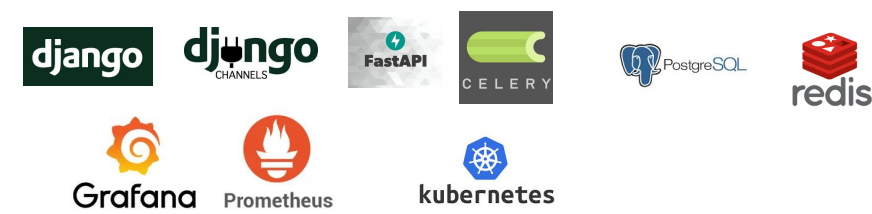
프로젝트 개요

SenSa는 산업현장의 가스·전력 센서와 작업자 위치를 실시간으로 수집하여, 위험 상태를 판정하고, 위험구역의 확산을 예측하며, 관제 화면과 외부 채널로 경보를 전달하는 통합 모니터링 플랫폼

목표: 센서 데이터가 유입된 시점부터 화면 표시, 경보 발행까지의 경로를 지연없이, 부하를 통제하며 처리하는것. 상태판정의 즉시성, 경보의 신뢰성, 운영 안정성을 동시에 달성하는것을 설계 기준으로 삼음.

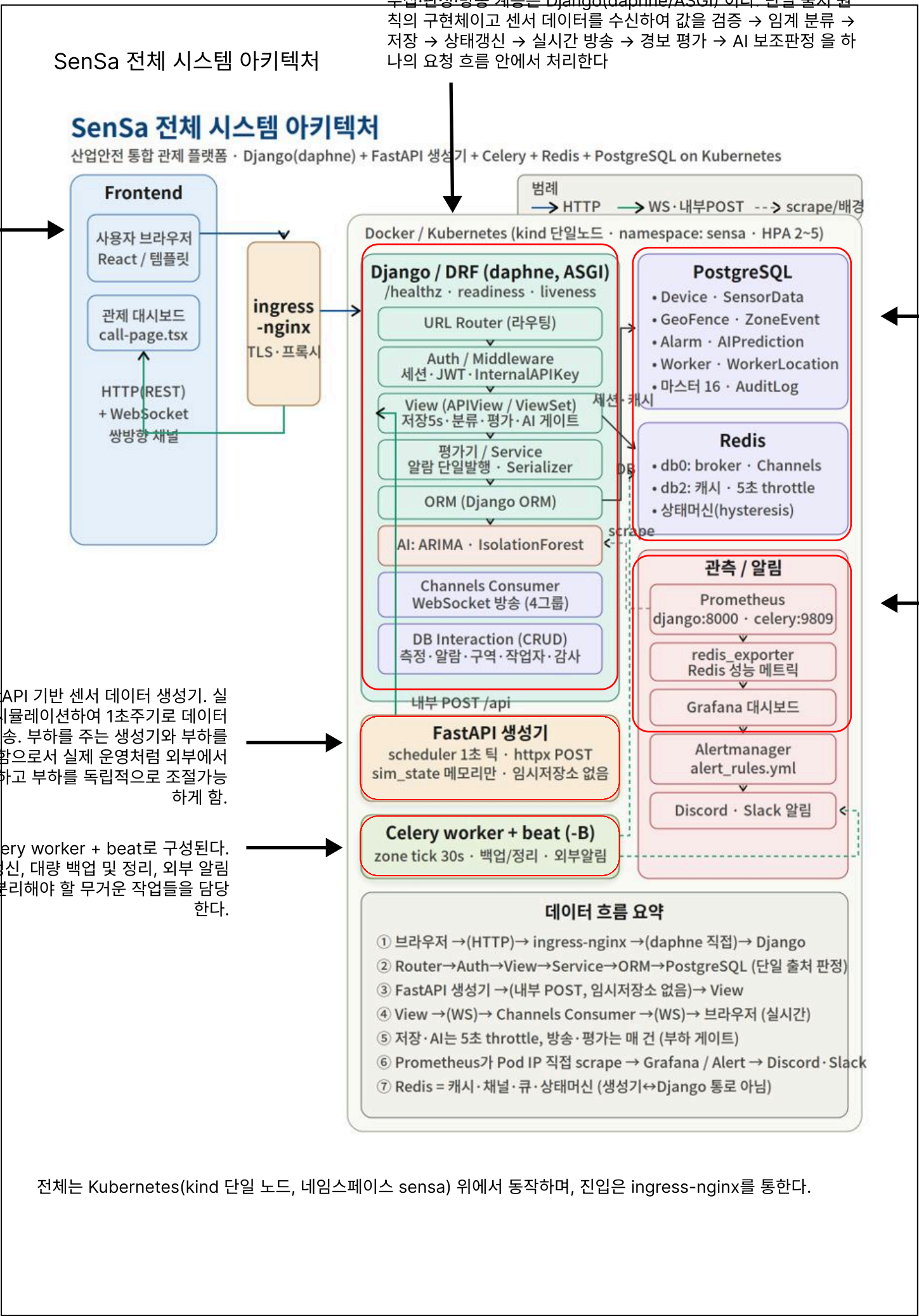
기간 2026.03~2026.06

핵심 스택

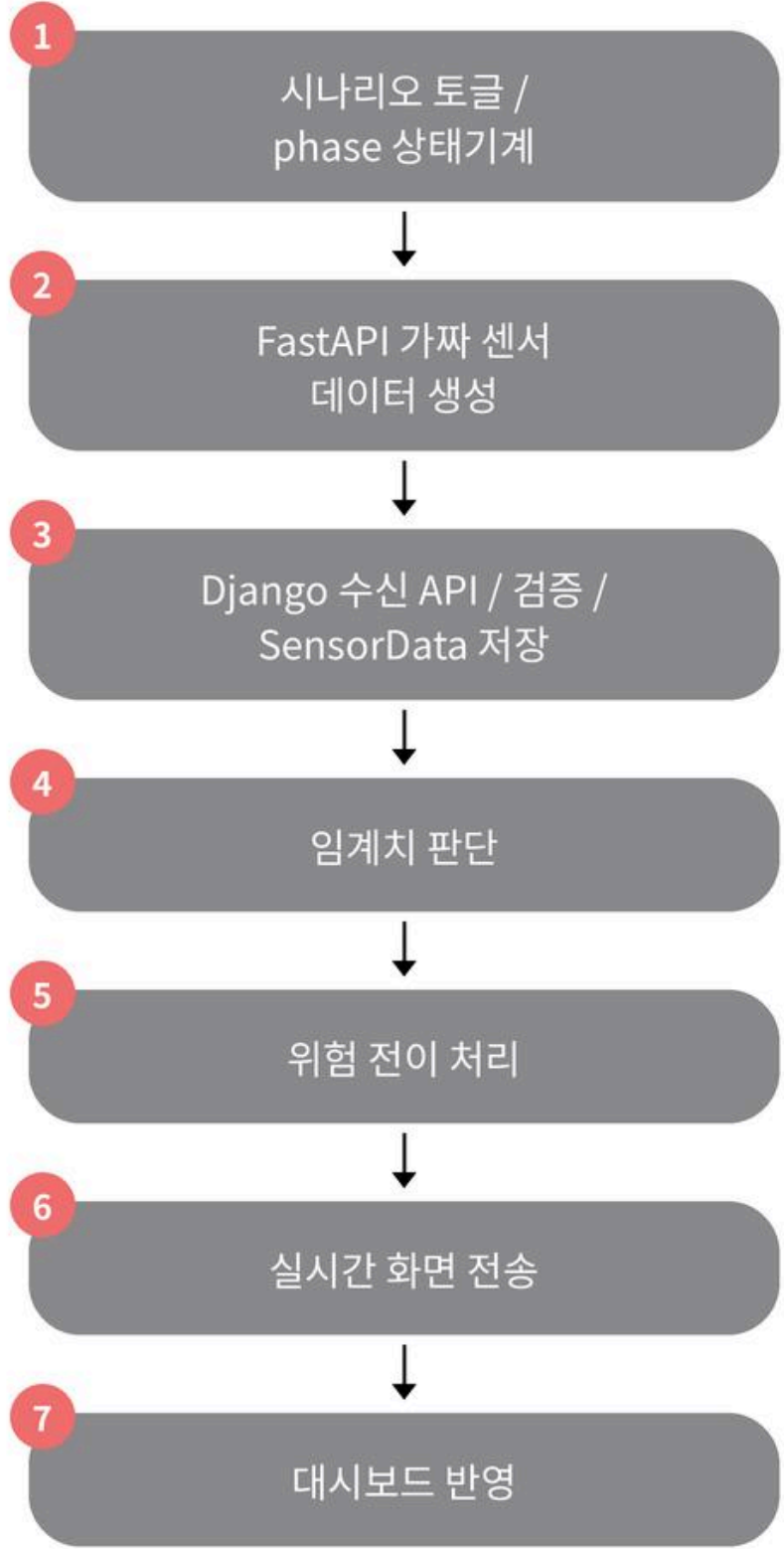


영역	기술	선택 근거
웹 프레임워크	Django +DRF	도메인 모델·관리자·인증이 풍부하고, 단일 도메인·소규모(센서 약 10대)에 모놀리식이 운영 복잡도 면에서 합리적
애플리케이션 서버	daphne(ASGI)	WebSocket 실시간 push가 핵심 요구 사항이라 HTTP 만 다루는 WSGI로는 불가능
실시간	Django Channels	WebSocket 그룹 브로드캐스트를 표준적으로 제공, Redis 채널 레이어와 결합
데이터 생성	FastAPI	생성기를 별도 서비스로 분리해 부하를 독립 조절, 비동기 HTTP(httpx)로 다수 센서 동시 전송
비동기 작업	Celery (worker + beat)	주기 작업(위험구역 tick)과 오프로드 작업(백업·외부알림)을 요청 경로에서 분리
캐시·상태·큐	Redis	부하 게이트의 원자 연산, 상태머신, 채널 레이어, 작업 broker를 단일
영속 저장	PostgreSQL	시계열 데이터의 인덱스·집계·트랜잭션 처리
컨테이너·오케스트레이션	Docker · Kubernetes(kind)	무중단 배포·자가복구·HPA를 학습·시연하고, 동일 매니페스트로 클라우드 전환 가능
관측	Prometheus · Grafana	pull 기반 메트릭 수집과 대시보드, 비즈니스 메트릭(경보·억제 건수)까지 노출

시스템 아키텍처



중앙 흐름도



데이터 수집, 판정 파이프라인(핵심)

데이터 수집, 판정 파이프라인

데이터 파이프라인 — 센서 1건이 화면에 뜨기까지



센서 1건이 들어와 화면에 표시되기까지의 직선 흐름

생성기에서 수신구까지

임시 저장소 없는 직접 전송 — 큐·버퍼 없이 httpx POST 한 번으로 끝나는 경로



설계 포인트

무상태 생성기

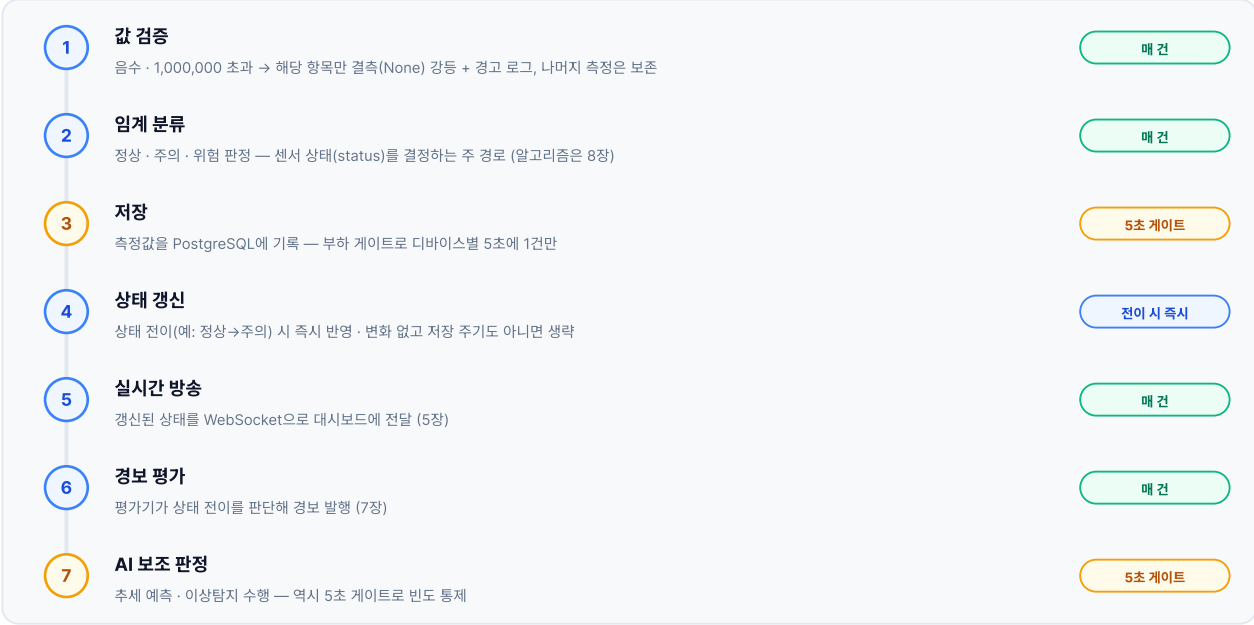
생성기는 들어오는 데이터를 쌓지 않는다.
sim_state는 생성 로직 내부의 보조 메모리일 뿐.

내부 서비스 인증

생성기 ↔ Django 사이의 내부 통신은
X-Internal-API-Key 헤더로 인증한다.

수신구 내부 처리 순서

SensorDataView.post() — 한 건의 측정이 거치는 7단계



“5초 vs 매 건” — 부하와 즉시성의 분리

무거운 작업과 가벼운 작업을 서로 다른 주기로 처리하는 파이프라인의 핵심 설계

매 건 수행 — 즉시성 유지

안전상 중요한 판정은 한 건도 지연 없이 처리

상태 판정 (정상 · 주의 · 위험)

실시간 방송 (WebSocket)

경보 평가

가볍고, 안전에 직결되는 경로

5초에 1건 — 부하 통제

초당 유입에 비례해 폭증할 수 있는 작업은 게이트로 묶는다

DB 저장 (PostgreSQL)

DB_SAVE_INTERVAL_SEC = 5

AI 추론 (추세 예측 · 이상탐지)

AI_INFERENCE_INTERVAL_SEC = 5

무겁고, 유입량에 비례해 커지는 경로 — 부하 1/5

틱 타임라인 (디바이스 1대 기준)

매 건 — 상태 판정 · 방송 · 경보 평가

판정 즉시성 100% · 저장 · 추론 부하 1/5 ↓



5초 게이트 — DB 저장 · AI 추론

실시간 처리 (websocket / channels)

실시간 처리 (WebSocket / Channels)
대시보드에서 새로고침 없이 센서 상태, 작업자 위치, 경보, 위험구역 변화를 실시간으로 받기 위해서 Django channels의 WebSocket cosumer를 사용함.

브라우저당 하나의 WebSocket 연결로 네 개의 그룹(경보·작업자·센서·위험구역)을 동시 구독함. 연결 수를 최소화하기 위한 의도적 설계다. 연결 수립 시점에 인증을 확인하여 미인증 연결은 수락 전에 차단하고, ASGI 진입부에서 Origin 검증과 세션 인증 스택으로 한 번 더 감싼다. 실시간 채널의 인증 누락은 흔한 취약점인데, 본 시스템은 이를 막고 있다.

발행은 별도의 래퍼 계층을 통한다. 호출자(수신구·평가기·위험구역 이벤트)가 채널 레이어 API를 직접 다루지 않게 감추어, 발행 경로를 단일 통로로 만들었다. 이 통로의 핵심은 장애 격리다. 메시지 채널(Redis) 장애 시 발생하는 예외를 광범위하게 흡수하고 경고 로그만 남긴다. 산업안전 시스템에서 메시지 큐 장애가 원본 경로(DB 저장·HTTP 응답)를 죽여 측정값이 영구 손실되는 것은 본말전도이기 때문이다. 방송 실패는 후속 방송이나 클라이언트 재연결로 자연 복구되지만, 저장 실패는 복구되지 않는다. 또한 방송과 동시에 “최신 상태 캐시”를 갱신하여, 다음 사용자가 화면을 처음 열 때 DB 조회 없이 마지막 상태를 즉시 그릴 수 있게 했다.

실시간 처리 아키텍처

SenSa

WebSocket · Django Channels — 센서 상태 · 작업자 위치 · 경보 · 위험구역 실시간 갱신



경보 로직

경보는 SenSa에서 가장 정교한 제어가 필요한 부분이다. 너무 자주 올리면 경보 피로를 유발하고, 누락되면 안전 사고를 부르기 때문

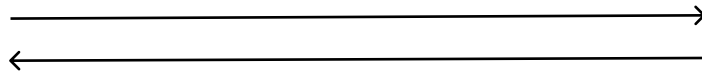
히스테리시스 - 떨림 방지

상태가 임계 근처에서 출렁일때 경보가 커졌다 꺼졌다 반복하면서 폭주하지 않도록 히스테리시스를 적용함. 악화방향 (예: 주의 → 위험)은 안전을 위해 즉시 반영하되 회복 방향은 보수적으로 처리함



info

악화: 일정시간(10초)안에 주의,위험 확정(5회 누적 요함)



회복: 일정시간(10초) 창안에서 정상신호 7회 누적 요함



danger

회복 임계(7회)가 격상 임계(5회)보다 높다는 것은 “악화는 민감하게, 회복은 신중하게” 라는 안전 도메인의 사상이 수치에 반영된것

독립 윈도우 카운터 — V자 진동 차단

독립 윈도우 카운터

SenSa

평가는 관측 상태별로 독립적인 카운터를 유지해 V자 진동을 차단한다

문제 — 신호 오염

서로 다른 소스의 신호가 엇갈려 들어온다

Celery → 위험 신호

생성기 → 정상 신호

단일 카운터 → V자 진동

한쪽 신호가 다른 쪽 진행을 오염시킨다

해법 — 카운터 분리

격상 · 회복을 각자의 독립 카운터로 누적

현재 공식 상태

격상 카운터 ↑

위험 방향 진행을 독립적으로 누적

회복 카운터 ↓

정상 방향 진행을 독립적으로 누적

서로의 진행을 침범하지 않는다

관측 상태 = 현재 공식 상태일 때의 규칙

관측된 상태가 지금의 공식 상태와 같으면, 반대 방향(격하)의 카운터만 0으로 초기화한다.

이때 진행 중인 격상 카운트는 보존되어, 같은 상태가 잠깐 섞여 들어와도 격상 흐름이 끊기지 않는다.

→ 격상은 살리고 격하만 리셋 · 격상과 회복은 끝까지 독립적으로 진행된다.

단일 발행과 재알림 억제

경보 발행은 평가기 한 곳에서만 일어남. 과거에는 평가기 외의 호출자도 발행하여 동일한 경보 토스트가 두번 뜨던 버그가 있었고, 이를 평가기 내부에서만 1회 발행으로 구조를 고정하여 해결함.

이상상태가 지속되는 경우 동일 경보는 60초 간격으로만 재발행함(RE_ALARM_INTERVAL_SEC =60). 중요한 점은 억제 되어 발행되지 않은 경보의 건수를 메트릭을 노출한다는 것임.

이를 통해 경보가 묻힌것이 아닌 의도적으로 줄인것이라는 점을 Grafana를 통해 수치로 증명 가능함. 경보 생성(alarm_created_total)과 억제(alarm)throttled_total)을 함께 카운트함.

AI 이상탐지

운영 AI는 두 가지 경량 시계열 모델로 구성됨

ARIMA ARIMA는 최근 윈도우(500개)의 측정값으로 학습하여 잔차의 3σ 를 벗어나는 값을 이상으로 탐지한다(최소 학습 샘플 60개).
ARIMA 차수는 *pmdarima*가 설치된 환경에서는 *auto_arima*로 최적 차수를 자동 탐색하며, *pmdarima*가 없을 때만 (1,1,1)을 폴백 값으로 사용한다.
*statsmodels*마저 없는 경우에는 이상 탐지를 수행하지 않고 *False*를 반환한다.
선형 외삽 *fallback*은 구현되지 않았으며, *arima_forecast.py*는 메인 파이프라인에서 호출되지 않는 미사용 파일이다.
가스의 경우 ARIMA 결과는 상태를 바꾸지 않고 보조 설명으로만 쓰이며, 전력의 경우에만 상태를 주의로 격상시킨다. 재학습은 별도 스레드(*threading.Thread*)에서 시간 기준으로 10분마다 (*REFIT_INTERVAL_SEC = 600*) 수행되며, 디바이스별 상태 저장소에는 스레드 락이 적용된다.
ARIMA 이상 탐지 자체에는 별도 *throttle*이 없으며, 히스테리시스 로직이 반복 경보를 자체적으로 억제한다.

Isolation Forest *IsolationForest*(트리 100개, *contamination 0.05*)는 추세를 예측하여 “이 추세라면 곧 임계를 초과할 것”을 선제적으로 경고한다(윈도우 200개, 최소 학습 70개).
예측 알람 발행 조건은 IF 단독이 아니라 두 조건을 모두 충족해야 한다(*trend_predictor.py:188*).
첫째, IF 이상 점수(*if_score < -0.3*) — IF가 현재 데이터 패턴을 이상으로 판단.
둘째, 선형 기울기(*slope ≥ 0.5*) — *numpy polyfit* 선형회귀가 값이 빠르게 상승 중임을 확인.
두 조건을 AND로 결합하는 이유는, IF만으로는 이상 여부는 알 수 있으나 상승 방향인지 알 수 없고, *slope*만으로는 상승은 확인하나 비정상 여부를 판단할 수 없기 때문이다.
즉 IF는 이상 탐지기, *slope*는 방향 필터 역할을 하여, 두 조건이 결합될 때 “비정상적으로 빠르게 상승 중” = 임계치 초과 임박을 예측할 수 있다.
학습 전 MAD(중앙값 절대편차) 기반으로 이상치를 제거하여 정상 분포만 학습하고, 재학습은 건수 기준으로 20 tick마다(*REFIT_EVERY_N = 20*) 수행된다.
사후 검증 루프(*verify_predictions*)가 예측 적중 여부를 기록하며, *AIPrediction* 모델에 메타데이터를 적재한다

선택의 근거 → 규모

센서 약 10대 규모에서는 무거운 딥러닝이나 *zero-shot* 파운데이션 모델보다, 가볍고 설명 가능한 통계 모델이 운영 비용 대비 효용이 높다. (초기 검증 단계에서 파운데이션 모델을 비교 평가했으나, 운영 의존성과 추론 부담이 이득을 상회하여 운영 스택에서 제외했다.)
본인은 이들을 명확한 함수 인터페이스(이상 여부를 반환하는 불리언, 예측 정보를 반환하는 사전)로 캡슐화하여, 수신 파이프라인이 알고리즘 내부를 알 필요 없이 결과만 소비하도록 통합했다.
AI 예측 경보 *throttle*(180초, *_AI_PRED_THROTTLE_SEC = 180.0*)은 *IsolationForest* 예측 알람에만 적용되며, ARIMA 이상 탐지에는 별도 *throttle*이 없다.
ARIMA는 히스테리시스 로직이 반복 경보를 자체적으로 억제하기 때문에 *throttle*이 불필요하다.
재학습 주기는 두 모델이 서로 다른 기준을 사용한다.
ARIMA는 시간 기준(*REFIT_INTERVAL_SEC = 600*, 10분마다)으로 재학습하며, *IsolationForest*는 건수 기준(*REFIT_EVERY_N = 20*, 20 tick마다)으로 재학습한다.
이는 ARIMA가 시계열 패턴 변화를 시간 단위로 추적하고, *IsolationForest*는 데이터 누적량에 따라 분포를 갱신하는 특성 차이에서 비롯된 설계다.

IsolationForest 예측 성능 튜닝 이력

총 9회의 파라미터 튜닝을 통해 *IsolationForest* + 선형회귀 이중 조건의 최적 설정을 탐색하였다. 튜닝 대상은 IF 이상 점수 임계치(*IF_SCORE*), 선형 기울기 임계치(*SLOPE*), 슬라이딩 윈도우 크기(*WINDOW*)이다.

단계	IF_SCORE	SLOPE	WINDOW	expires	Precision	Recall	F1	비고
임계치 1	-0.05	0.5	30	x2	27.0%	83.0%	43.0%	초기값
임계치 2	-0.20	0.5	30	x2	14.0%	38.2%	31.2%	IF 강화
임계치 3	-0.35	0.5	30	x2	7.0%	28.0%	48.0%	과도한 엄격화
기울기 1	-0.35	0.7	50	x2	10.0%	34.0%	29.0%	SLOPE 강화
기울기 2	-0.35	0.5	50	x2	36.0%	32.0%	29.0%	SLOPE 기준
기울기3 ★	-0.35	0.3	50	x2	41.0%	40.0%	32.0%	SLOPE 완화
윈도우 1	-0.35	0.5	30	x2	25.0%	20.0%	16.0%	WINDOW 기준
윈도우 2 ★	-0.35	0.5	15	x2	38.0%	38.0%	31.0%	WINDOW 축소
시나리오 ★★	-0.05	0.5	50	x2	82.2%	40.0%	0.4%	실제 패턴

★ 균형적 성능 개선 ★★ 최고 Precision (실제 상승 패턴 적용 시)

파라미터 상관관계 분석 (피어슨 r)

9회 튜닝 결과를 바탕으로 파라미터와 성능 지표 간의 피어슨 상관계수(r)를 산출하였다. $|r| \geq 0.7$ 이면 강한 상관, 0.4~0.7이면 중간 상관, 0.4 미만이면 약한 상관으로 분류한다.

파라미터 A	파라미터 B	r 값	강도/방향	해석
IF_SCORE	Precision	+0.477	중간 양의 상관	덜 엄격할수록 Precision 상승
IF_SCORE	Recall	+0.706	강한 양의 상관	덜 엄격할수록 Recall 큰 폭 상승
IF_SCORE	F1	-0.266	약한 음의 상관	IF_SCORE와 F1은 약한 반비례
SLOPE	Precision	-0.340	약한 음의 상관	SLOPE 낮을수록 Precision 상승 경향
SLOPE	Recall	-0.085	약한 음의 상관	SLOPE와 Recall 상관 거의 없음
SLOPE	F1	-0.054	약한 음의 상관	SLOPE와 F1 상관 거의 없음
WINDOW	Precision	+0.333	약한 양의 상관	WINDOW와 Precision 약한 비례
WINDOW	Recall	-0.109	약한 음의 상관	WINDOW와 Recall 상관 미미
WINDOW	F1	-0.366	약한 음의 상관	WINDOW 클수록 F1 약간 하락
Precision	Recall	+0.105	약한 양의 상관	Precision과 Recall 상관 거의 없음
Precision	F1	-0.743	강한 음의 상관	Precision 높으면 F1 낮아지는 강한 반비례
Recall	F1	+0.317	약한 양의 상관	Recall과 F1 약한 비례

주요 발견 및 권장 설정

IF_SCORE ↔ *Recall* ($r = +0.706$) — 강한 양의 상관: *IF_SCORE*가 높아질수록(덜 엄격) *Recall*도 높아지는 강한 비례 관계. -0.05에서 *Recall* 83%, -0.35에서 20~34%로 하락. IF를 엄격하게 조일수록 예측 자체를 안 하게 되어 *Recall*이 급락한다.

Precision ↔ *F1* ($r = -0.743$) — 강한 음의 상관: *Precision*이 높아질수록 *F1*이 낮아지는 강한 반비례 관계. 시나리오 적용 시 *Precision* 82.2%이지만 *F1*은 0.4%로 급락. *Recall*이 낮은 상태에서 *Precision*만 높아지면 *F1* 조화평균이 크게 낮아지기 때문이다.

IF_SCORE ↔ *Precision* ($r = +0.477$) — 중간 양의 상관: 엄격하게 조일수록 *Precision*이 오히려 낮아지는 역설적 결과. -0.35(엄격)에서 *Precision* 7.0%~41.0%로 불안정. 엄격한 조건에서 예측 수가 줄어들어 통계적으로 불안정해지기 때문이다.

SLOPE ↔ *Precision* ($r = -0.340$) — 약한 음의 상관: *SLOPE*가 낮을수록(완만한 기울기도 허용) *Precision* 상승 경향. *SLOPE*=0.3에서 P 41% · R 40% 동반 상승. *SLOPE*=0.7에서 *Precision* 10%로 하락.

파라미터	현재 경향	권장 방향	근거
IF_SCORE	낮을수록(엄격) Precision 하락	-0.05 유지 권장	엄격화 시 오히려 7%까지 하락; 덜 엄격 + 실제 패턴에서 82.2%
SLOPE	낮을수록(완만) P-R 동반 상승	0.3~0.5 탐색 권장	0.3에서 P 41% · R 40% 균형 달성
WINDOW	작을수록 P-R 균형 개선	15~30 탐색 권장	15에서 P 38% · R 38% 가장 균형적
운영 환경	실제 패턴이 랜덤보다 훨씬 유리	시나리오 기반 검증 필수	랜덤 최고 41% → 시나리오 82.2%

종합 결론

*IsolationForest*의 임계치(*IF_SCORE*)를 엄격하게 조이는 것만으로는 성능이 개선되지 않는다. 오히려 덜 엄격한 조건(-0.05)에서 *SLOPE*를 완화(0.3)하고 *WINDOW*를 줄이는(15) 방식이 *Precision*과 *Recall*을 균형 있게 개선하는 데 더 효과적이다.
무엇보다 실제 상승 패턴(시나리오 환경)에서 측정한 결과가 랜덤 데이터 대비 훨씬 높은 성능을 보여, 운영 환경 기반의 검증이 필수적임을 확인하였다.

데이터 모델

SensorData(측정 시계열)는 가스 9종·전력 3종·온도를 한 테이블에 담는다. 측정 종류가 고정·소수이므로, 조인 없이 단순하고 빠른 조회를 위해 결측이 많은(*sparse*) 넓은 테이블을 의도적으로 택했다. “특정 센서의 최신 *N*건” 조회(대시보드·AI 윈도우)가 빈번하므로 (디바이스, 시각 역순) 복합 인덱스를 두었다.

Device(센서)는 최신 상태와 값을 비정규화로 보유한다. 매번 *SensorData*를 집계하지 않고 *Device* 행에서 직접 읽어 목록 조회를 빠르게 하되, 상태 전이 시 즉시 갱신하여 *stale*을 막는다.

GeoFence(위험구역)는 정적(관리자 *polygon*)과 동적(자동 확산) 구역을 한 모델로 통합한다. *polygon*이 두 경우 모두 동일한 좌표 배열 형식이라 작업자 진입 판정 로직을 공유할 수 있다. 센서와는 세 가지 관계 — 소속(센서가 속한 구역), 발원(구역을 만든 센서), 확인(확산을 확인한 센서들의 다대다) — 을 가진다.

*Alarm*과 *AIPrediction*은 1:1로 연결되어, AI 예측 경보의 메타데이터와 사후 검증 결과를 추적할 수 있게 한다.

*USE_TZ*를 활성화하여 모든 시각을 타임존 인식(*aware*)으로 저장하며, 이는 여러 모델을 시간순으로 병합하는 통합 로그의 정렬 일관성을 보장한다

SenSa_ERD.mermaid / SenSa_ERD_설명.md 파일의 erd 참고

알림 체계

알림 디스패치 파이프라인 — 사내(In-App) 알림

Alarm 생성 시그널이 트리거. Celery 아닌 daemon thread 큐. 외부 Discord(geofence)와 별개 경로 · backoffice/



핵심 포인트 · 데이터/외부알림과 다른 점

- 트리거가 “센서 1건”이 아니라 Alarm 생성 시그널 — 알람이 생기면 자동으로 따라붙는 후속 경로
- Celery가 아닌 daemon thread 큐(BACKOFFICE_ASYNC_NOTIFY) — 가법계, 단 프로세스당 1워커(본산 단순화)
- 이진 사내(In-App) 알림(NotificationLog). 외부 Discord 위험 알람은 geofence→notifiers(Celery)로 완전히 다른 경로
- 수신자 DSL · 정책 우선순위 매핑이 핵심 (누구에게 보낼지)
- 발송 채널(FCM/SMS)은 stub — 후속 연동 지점

SenSa는 두 개의 독립된 알람 경로를 가짐

외부 위험 알람(Discord) → 위험구역 이벤트에서 발생. critical 승격 또는 동적 위험구역 생성 시(주의 등급은 제외) 단일 발행점에서 외부 알람 태스크를 Celery로 큐잉한다.

동기 webhook 호출(최대 5초 타임아웃)이 위험구역 생애주기를 막지 않게 하기 위해 비동기로 분리했다.

사내 알람(In-App) → 경보 생성 시그널에서 발생. 경보가 저장되면 자동으로 디스패처가 동작하여, 위험 범주를 매핑하고 알람 정책을 조회한 뒤, 수신자 토큰DSL(전체 사용자, 리더, 그룹, 역할)로 대상을 풀어 사용자x채널별 알람 로그를 생성함.

이 경로는 Celery가 아닌 데몬 스레드 큐로 가볍게 처리한다(실제 FCM/SMS 발송 채널은 추상화한 완료된 자리표시 상태)

감사 로그 파이프라인 — 누가·언제·무엇을 (Audit)

요청마다 thread-local로 actor 주입 + 모델 시그널 자동 기록. 백오피스 변경 추적 · backoffice/



핵심 포인트

- 횡단 관심사(누가·어디서)를 thread-local로 시그널에 주입 — 뷰 코드를 건드리지 않고 자동 기록
- allowlist 16모델만 추적 — SensorData·WorkerLocation 같은 초당 트래픽은 제외(감사 폭증 방지)
- actor_username 스냅샷 + 시드 제외 가드 = “사용자가 한 일만, 사후에도 추적 가능”
- 데이터 파이프라인과 무관 — 관리 행위 추적을 독립 경로
- 감사 실패는 try/except로 흡수(본 작업 보호)

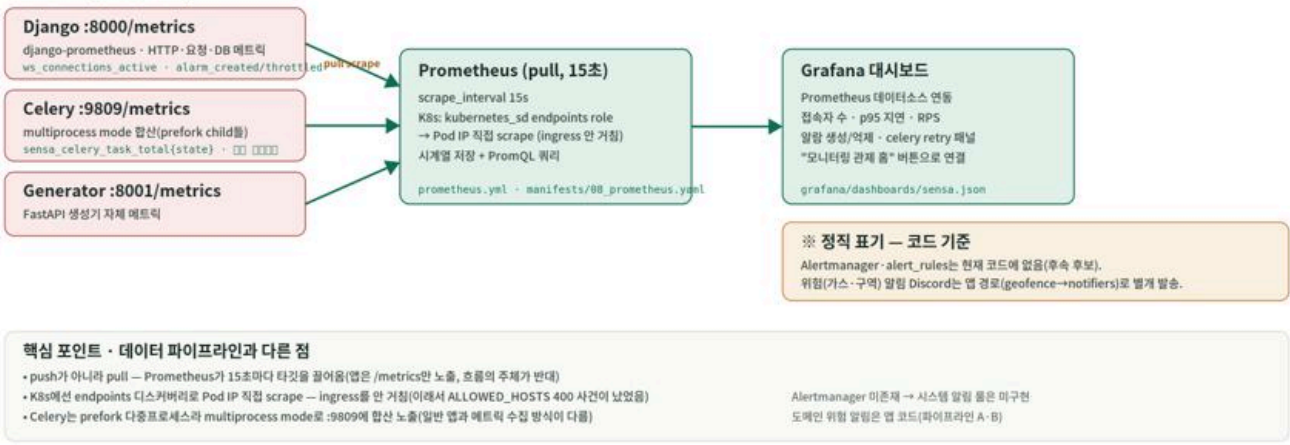
관리 행위는 별도의 감사 경로로 추적. 미들웨어가 요청 시작시 요청 객체를 thread-local에 주입하여, 모델 시그널이 “누가, 어디서” 변경했는지 알 수 있게 한다. 추적 대상은 16개의 핵심 모델로 한정(allowlist)하여, 초당 유입되는 측정, 위치 데이터는 감사에서 제외함으로서 감사 로그 폭증을 막는다. 마이그레이션 시드처럼 요청이 없는 변경은 감사하지 않으며, 행위자 이름을 스냅샷으로 저장하여 사용자가 이후 삭제, 개명되어도 기록이 보존된다.

관측 및 운영

관측 파이프라인 — Prometheus scrape → Grafana

Prometheus가 각 타겟을 주기적으로 끌어옴(pull), 코드 기준 Alertmanager 없음 — 위험 알람은 앱 경로(파이프라인 A·B) · sensa_observability/

메트릭 소스 (scrape 타겟)



관측 —————> pull 방식이고 Prometheus가 15초 주기로 각 타겟-Django(:8000), 생성기(:8001), Celery(:9809)의 메트릭을 끌어온다. 애플리케이션은 /metrics를 노출만 하며, 수집의 주체는 Prometheus다.

Kubernetes 환경에서는 엔드포인트 서비스 디스커버리를 통해 Pod IP를 직접 scrape한다.

Celery —————> prefork 다중 프로세스 모델이라, 일반 메트릭 수집으로는 자식 프로세스의 카운터가 합산되지 않는다. 이를 multiprocess mode로 구성하여 각 자식이 공유 디렉토리에 기록하고 메인인 합산하여 노출하도록 했다.

Grafana —————> 접속자 수·응답 지연(p95)·처리량(RPS)·경보 생성/억제·Celery 재시도 등의 패널을 제공한다.

특히 경보 억제 건수와 Celery 재시도 건수를 노출하여, 부하 절감과 자동 재시도 정책이 실제로 작동했음을 정량적으로 보인다.

솔직한 기술 표기: 현재 코드에는 Alertmanager와 알람 규칙(alert_rules)이 포함되어 있지 않다. 관측은 Prometheus → Grafana까지이며, 위험(가스·구역) 경보의 외부 발송은 Alertmanager가 아니라 애플리케이션 코드(11장)에서 이루어진다. 시스템 레벨 알람 규칙(서비스 다운·오류율·지연)은 향후 과제다

팀원별 기여 및 역할 경계

김재승	백오피스 공통(공동), ARIMA 기반 이상탐지(공동), AI 학습 기반(공동), 데이터 저장 최적화 및 저장 주기 관리, 운영 데이터 관리(백업/복원/초기화), 알람 체계 및 시나리오 로직 설계(전원 공동).
최서원	계정/인증 시스템, 공통 헤더 및 사이드메뉴, 대시보드 작업자 현황(공동), 나의 안전확인(section 08), 작업자 현황 페이지, 안전 체크리스트, VR 교육, 내 정보(프로필), 실시간 통신 인프라(WebSocket / Redis / Django Channels), 컨테이너화 및 배포(Docker, Kubernetes, Grafana), DB 마이그레이션 및 시드 데이터 관리.
서호준	대시보드 지도(section 09), 대시보드 작업자 현황(공동), 대시보드 유해가스(공동), AI 예측 성능(section 16), 알람 기록 페이지, AI 알고리즘 개발, 알람 생성(공동), ARIMA 기반 이상탐지(공동), 백오피스 공통(공동).
이준호	대시보드 이벤트 현황(section 10), 대시보드 유해가스(공동), 대시보드 스마트 전력(section 14·15), ARIMA 기반 이상탐지(공동), AI 학습 기반(공동), 알람 생성(공동), 백오피스 공통(공동).
공동 담당 (전원)	알람 체계 및 시나리오 로직 설계. 각 팀원이 담당한 알고리즘은 명확한 함수 인터페이스로 캡슐화되어 있어, 파이프라인은 알고리즘 내부를 알 필요 없이 결과(상태 분류, 이상 여부, 예측 정보)만 소비하도록 설계되었다. 이 캡슐화 경계 설계는 팀 전체의 협업 구조다.

시나리오 동작 검증

시나리오 동작 검증 (검증 ①) — End-to-End)

이 설계는 검증 ③(14장)에서 실제 운영 구성(Kubernetes, 다중 replica) 기준으로 end-to-end 확인되었다

시나리오 명령 한 번이 데이터 생성 → 임계 분류(의도 대비 일치를 95.2%, n=1,040, 순수 오분류 0건) → 화면·위험구역 반 응 → 정상 복귀로 이어짐을 단계별 증빙으로 검증했다.

별도 수행한 검증 ③(측정 일시 2026-06-11, 실제 운영 구성 Kubernetes-네임스페이스 sensa)의 결과를 통합한 것이다.

스크린샷 증빙은 본문 그림(그림 1~7)과의 번호 출몰을 피하기 위해 “검③ 그림 N”으로 표기한다.

요약

본 검증은 시나리오 전환 명령 한 번이 데이터 생성 → 임계 분류 → 화면·경보·위험구역 반응으로 이어지는 전체 인과 사슬이 설계대로 작동함을 실제 운영 환경에서 확인한 것이다.

첫째, 시나리오 전환 명령이 즉시·정확하게 상태기계에 반영되는가.

둘째, 각 phase에서 생성되는 데이터가 설계된 분포(목표 평균·표준편차)를 따르는가.

셋째, 그 데이터가 수신 파이프라인을 거쳐 분류·표시·경보·위험구역까지 올바르게 이어지는가.

검증 항목	결과	증빙
시나리오 상태기계	토글 1회로 phase 1→2→3→4 자동 진행 (30초/phase, 설계값 일치)	검③ 그림 1, 2
데이터 생성 변화	H ₂ S 표시 ~3ppm → 위험선 (50ppm) 돌파 → peak 유지	검③ 그림 3
채널 독립성	대상 채널(H ₂ S)만 변화, 비영향 8 중 채널은 정상 유지	검③ 그림 4
수신측 임계 분류	H ₂ S 63.94ppm → ‘위험’ 판정 (임계치 50ppm 기준 정확)	검③ 그림 4
통합 화면 반응	관계 대시보드 위험 표시·위험구역 생성·실시간 수치 반영	검③ 그림 5, 6
회복 사이클	토글 OFF → phase 5→0 자동 복귀, H ₂ S baseline 하강·‘정상’ 재분류	검③ 그림 7~9
의도 대비 분류 정합	expected_status vs 실제 분류 일치율 95.2% (n=1,040), 순수 오분류 0건	검③ 그림 10, 11

검증 목적과 범위

SenSa의 시연-검증 체계는 합성 데이터 생성기가 현실적인 사고 시나리오를 재현할 수 있는지에 의존한다. 따라서 본 검증의 질문은 다음 세 가지다.

첫째, 시나리오 전환 명령이 즉시·정확하게 상태기계에 반영되는가.

둘째, 각 phase에서 생성되는 데이터가 설계된 분포(목표 평균·표준편차)를 따르는가.

셋째, 그 데이터가 수신 파이프라인을 거쳐 분류·표시·경보·위험구역까지 올바르게 이어지는가.

검증 범위는 R&D 시나리오 토클 시스템(5개 device 매핑: G3 H₂S 누출, G4 CO 연소이상, G1 환기 불량, P1 과부하, P3 전압 강하) 중 G3(sensor_01, H₂S 누출) 시나리오를 대표로 한 end-to-end 흐름이다.

단일 채널만 변화는 시나리오와 인과 관계가 그래프에서 가장 명확하게 드러나기 때문에 대표로 선정하였다.

검증 환경과 대상 매커니즘

검증은 별도의 모의 환경이 아닌 실제 운영 구성에서 수행하였다: kind 단일 노드 Kubernetes(네임스페이스 sensa) 위에서 Django(daphne/ASGI, HPA 2~5 replica), FastAPI 데이터 생성기, Celery worker/beat, Redis, PostgreSQL, Prometheus/Grafana가 모두 가동 중인 상태였다. 검증 중 실제로 Django pod가 3개로 스케일아웃된 다중 replica 환경 이었다.

검증 대상 매커니즘(코드 기준): 생성기는 POST /anomaly/toggle?device_id=...&state=true 명령을 받으면 해당 device의 phase 상태기계를 기동한다.

phase는 0(정상)→1(진조)→2(주의)→3(위험)→4(조치 peak)로 가스 기준 30초마다 자동 전환되며, 각 phase는 목표 평균·표준편차를 갖고 OU(Ornstein-Uhlenbeck) 평균회귀 노이즈로 현실적인 시계열을 생성한다.

G3 시나리오의 H₂S 목표값

Phase	단계	H2S 목표 (μ, σ)	라벨 (expected_status)
0	정상	3.1 ± 1.5 ppm	normal
1	전조	7 ± 2 ppm	normal (AI lead-time 평가용)
2	주의	18 ± 3 ppm	caution
3	위험	65 ± 8 ppm	danger
4	조치(peak)	70 ± 8 ppm	danger
5	복귀	8 ± 3 ppm	normal

생성된 매 데이터에는 scenario_id, expected_phase, expected_status 라벨이 동봉되어 Django SensorData 에 설계가 저장된다. 즉 “생성기가 의도한 상태”와 “수신측이 실제 분류한 상태”를 사후 대조할 수 있는 구조다.

검증 절차와 결과

사전 상태(Baseline) 확인

개입 전 상태를 먼저 고정한다.

생성기 상태 조회(GET /)와 토글 상태 조회(GET /anomaly/state) 결과, 시나리오는 normal, 매핑된 5개 device 전부 토글 OFF-phase 00이다.

이 시점의 H₂S는 검③ 그림 3의 좌측 구간에서 보통 평시 분포(약 3ppm 내외)를 유지하고 있었다.

```
curl -s localhost:8001/ | python3 -m json.tool
curl -s localhost:8001/anomaly/state | python3 -m json.tool
{
  "status": "running",
  "scenario": "normal",
  "tick interval": 1.0,
  "valid_scenarios": [
    "normal",
    "mixed",
    "danger"
  ]
}
{
  "phase_duration": {
    "gas": 30,
    "power": 120
  },
  "devices": {
    "sensor_01": {
      "on": false,
      "phase": 0,
      "elapsed_in_phase_sec": 0,
      "scenario_id": "G3"
    },
    "sensor_02": {
      "on": false,
      "phase": 0,
      "elapsed_in_phase_sec": 0,
      "scenario_id": "G4"
    },
    "sensor_03": {
      "on": false,
      "phase": 0,
      "elapsed_in_phase_sec": 0,
      "scenario_id": "G1"
    },
    "power_01": {
      "on": false,
      "phase": 0,
      "elapsed_in_phase_sec": 0,
      "scenario_id": "P1"
    },
    "power_02": {
      "on": false,
      "phase": 0,
      "elapsed_in_phase_sec": 0,
      "scenario_id": "P3"
    }
  }
}
```

Baseline - 시나리오 normal
5개 device 전부 OFFphase0

전환 명령 → phase 자동 진행

10:44에 단일 명령 POST /anomaly/toggle?device_id=sensor_01&state=true 를 실행하고, 15초 간격으로 상태를 추적 하였다. 결과는 설계와 정확히 일치했다.

명령 직후 phase 1로 전환되었고, 이후 추가 개입 없이 약 30초마다 phase 2(10:44:53) → phase 3(10:45:25) → phase 4(10:45:58)로 자동 전환되었으며, phase 4(peak)에서는 전환 없이 유지되었다.

즉 “명령 한 번 → 사고 시계열 자동 전개”라는 상태기계가 설계가 운영 환경에서 그대로 작동함을 확인했다.

```
○ (SenSa) normalframe1094@cs:~/SenSa$ # 토글 ON
curl -s -X POST 'localhost:8001/anomaly/toggle

# phase 진행 추적 (15초 × 12회 = 3분)
for i in $(seq 1 12); do
  echo "---- $(date +%T) ----"
  curl -s localhost:8001/anomaly/state | python3 -m json.tool
  sleep 15
done
{
  "ok": true,
  "device_id": "sensor_01",
  "on": true,
  "current_phase": 1,
  "phase_started_at": 1781142260.4673433
}
--- 10:44:20 ---
phase=1 elapsed=0s on=True
--- 10:44:35 ---
phase=1 elapsed=15s on=True
--- 10:44:53 ---
phase=2 elapsed=1s on=True
--- 10:45:08 ---
phase=2 elapsed=16s on=True
--- 10:45:25 ---
phase=3 elapsed=2s on=True
--- 10:45:40 ---
phase=3 elapsed=17s on=True
--- 10:45:58 ---
phase=4 elapsed=2s on=True
--- 10:46:13 ---
phase=4 elapsed=17s on=True
--- 10:46:31 ---
phase=4 elapsed=35s on=True
--- 10:46:46 ---
phase=4 elapsed=50s on=True
--- 10:47:04 ---
phase=4 elapsed=68s on=True
```

토글 ON 직후 15초 간격 추적
phase 1→2→3→4 자동 진행(각 30초),
peak 유지

데이터 생성 변화 — 시계열 확인

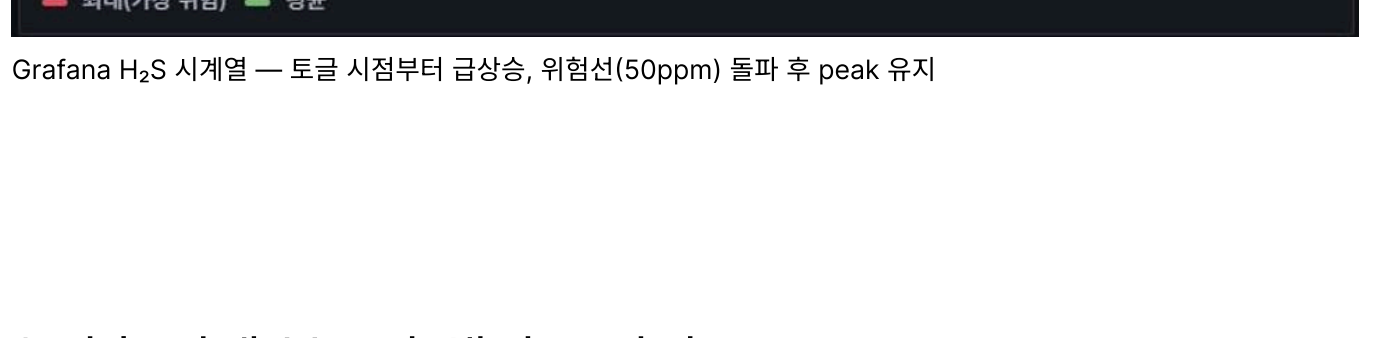
Grafana의 H₂S 패널(주의 10 / 위험 50 ppm 기준선 표시)에서 토글 전후를 비교하였다.

H₂S 63.94ppm이 ‘위험’으로 분류되었고(위험 임계 50ppm → 정확), 동시에 나머지 채널(O₂ 20.78%, CO 10.82ppm, CO₂ 559ppm, NO₂-SO₂ 등)는 전부 ‘정상’을 유지했다.

이는 두 가지를 동시에 증명한다.

첫째, 수신 서버의 임계 분류가 올바르게 동작한다.

둘째, G3 시나리오의 설계 원칙 — 대상 채널(H₂S)만 변화시키고 비영향 채널은 baseline + 노이즈로 유지 — 이 데이터 생성 단계에서 지켜지고 있다.



Grafana H₂S 시계열 — 토글 시점부터 급상승, 위험선(50ppm) 돌파 후 peak 유지

수신측 임계 분류와 채널 독립성

관계 대시보드의 유해가스 위험 현황 패널에서 수신 분류 결과를 확인하였다.

normal→caution 39건은 phase 1(전조)의 의도된 특성이다.

H₂S 63.94ppm이 ‘위험’으로 분류되었고(위험 임계 50ppm → 정확), 동시에 나머지 채널(O₂ 20.78%, CO 10.82ppm, CO₂ 559ppm, NO₂-SO₂ 등)는 전부 ‘정상’을 유지했다.

이는 두 가지를 동시에 증명한다.

첫째, 수신 서버의 임계 분류가 올바르게 동작한다.

둘째, G3 시나리오의 설계 원칙 — 대상 채널(H₂S)만 변화시키고 비영향 채널은 baseline + 노이즈로 유지 — 이 데이터 생성 단계에서 지켜지고 있다.

H ₂ S 관리주소			
H ₂ S 관리주소	63.94	ppm	위험
NO ₂ 이산화질소	0.05	ppm	정상
SO ₂ 이산화황	0.035	ppm	정상

수신측 분류 — H₂S만 '위험', 비영양 채널 전부 '정상' (채널 독립성 확인)

통합 화면 반응 — 위험구역 자동 발동 포함

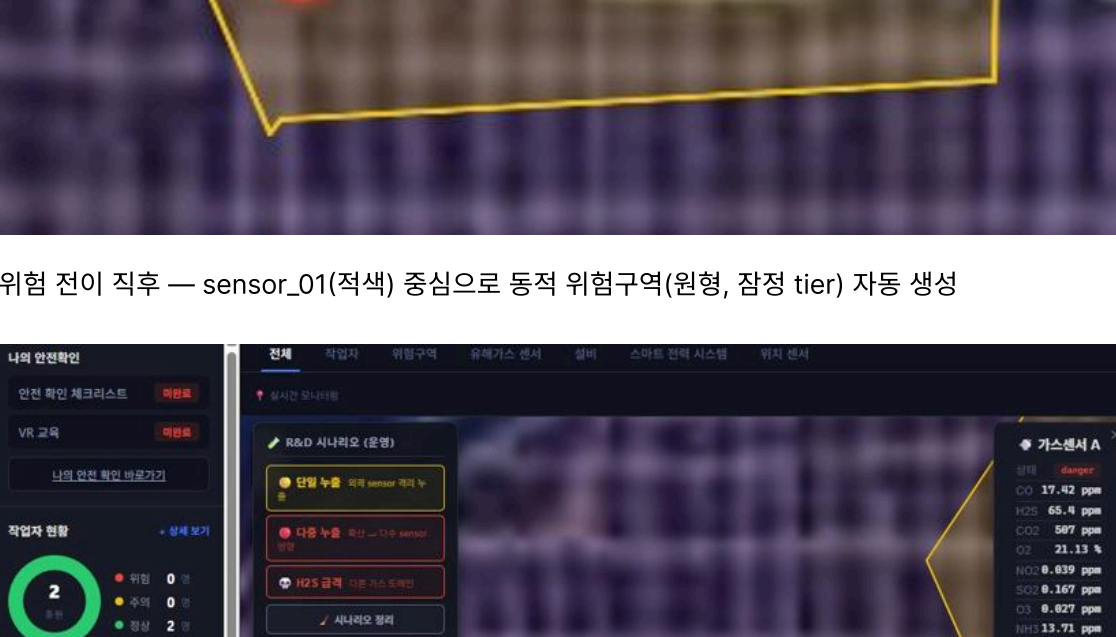
위험 phase 진입 후 관계 화면 전체의 반응을 확인하였다. 지도에서 sensor_010이 위험 상태로 표시되고, 해당 센서를 중심으로 동적 위험구역(잠정 tier, 원형 초기 반경)이 자동 생성되었다. 상세 팝업에는 H₂S 65.4ppm(phase 4 설계 분포 내)과 위험 배치가 실시간으로 반영되었으며, 차단 가스 현황 패널 시, 예측 패널도 동시 갱신되었다.

수신측 분류 — H₂S만 ‘위험’, 비영향 채널 전부 ‘정상’ (채널 독립성 확인)

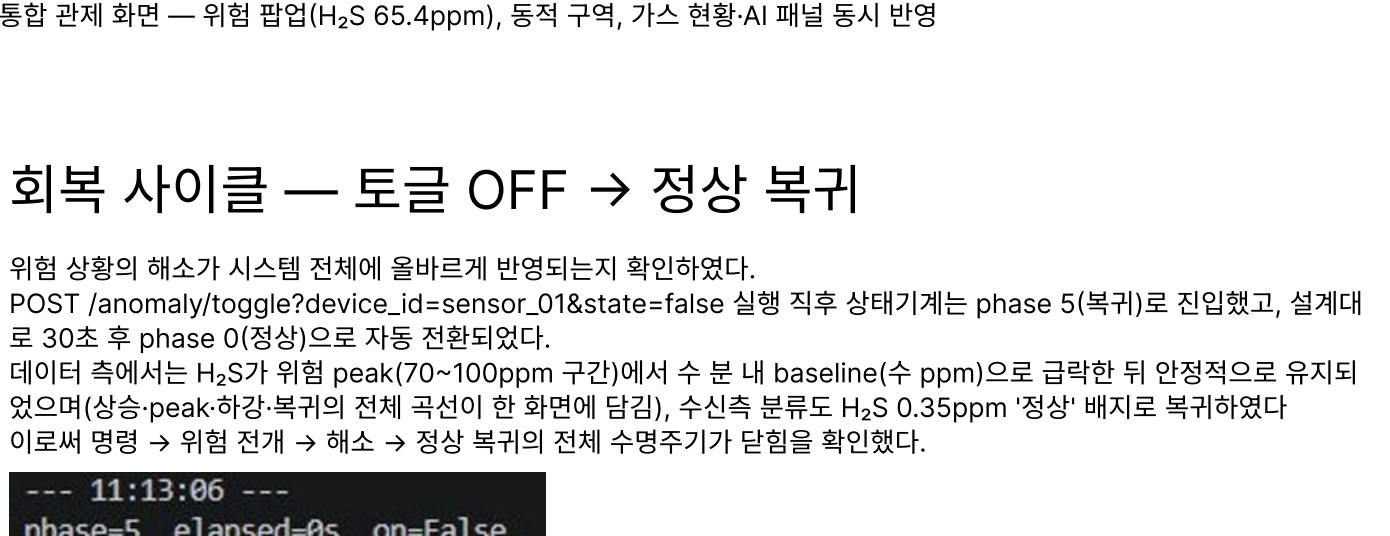
통합 화면 반응 — 위험구역 자동 발동 포함

위험 phase 진입 후 관계 화면 전체의 반응은 확인하였다. 지도에서 sensor_01이 위험 상태로 표시되고, 해당 센서를 중심으로 동적 위험구역(잠정 tier, 원형 초기 반경)이 자동 생성되었다. 상세 팝업에는 H₂S 65.4ppm(phase 4 설계 패널 내)과 위험 배치가 실시간으로 반영되었으며, 하단 가스 현황 패널AI 예측 패널도 동시 갱신되었다.

참고로 이 “임계 초과 → 위험구역 자동 발동” 경로는 본 검증 과정에서 발현한 설계 공백을 보완해 연결한 것이다



위험 전이 직후 — sensor_01(적색) 중심으로 동적 위험구역(원형, 잠정 tier) 자동 생성



회복 사이클 — 토글 OFF → 정상 복귀

위험 상황의 해소가 시스템 전체에 올바르게 반영되는지 확인하였다.

normal→caution 39건은 phase 1(전조)의 의도된 특성이다.

라벨은 normal이지만 목표 분포(7 ± 2ppm)가 주의 임계(10ppm)에 인접해 일부 표본이 임계를 넘도록 설계되어 있다 (AI가 이 구간을 조기 탐지하면 lead-time 성과로 평가하기 위함). danger→caution 10건과 caution→normal 1건은 phase 전환 직후 OU 평균회귀가 새 목표값에 도달하기까지의 약 3틱 ramp 구간에서 발생한 경계 지연이다.

이로써 명령 → 위험 전개 → 해소 → 정상 복귀의 전체 수명주기가 닫힘을 확인했다.

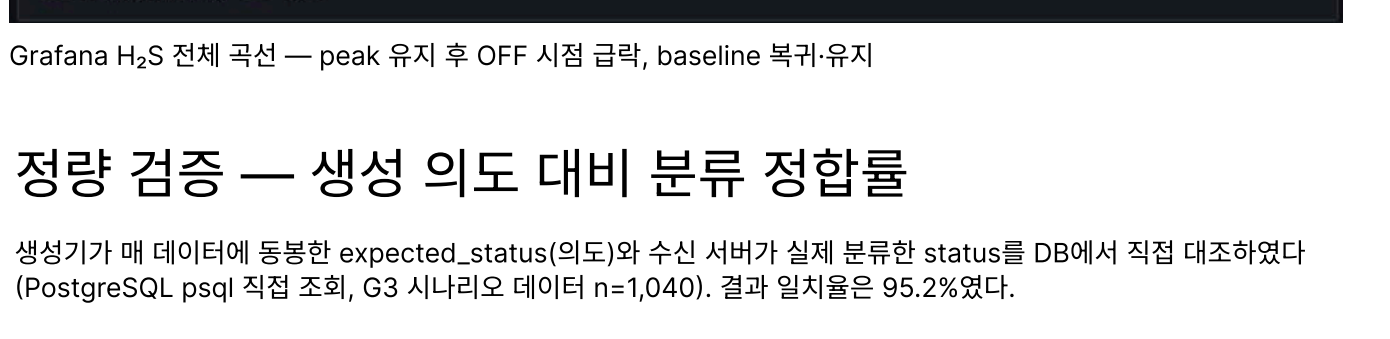
```
--- 11:13:06 ---
phase=5 elapsed=0s on=False
--- 11:13:23 ---
phase=5 elapsed=17s on=False
--- 11:13:38 ---
phase=0 elapsed=1s on=False
--- 11:13:56 ---
phase=0 elapsed=19s on=False
--- 11:14:11 ---
phase=0 elapsed=34s on=False
--- 11:14:29 ---
phase=0 elapsed=51s on=False
--- 11:14:44 ---
phase=0 elapsed=66s on=False
--- 11:15:02 ---
phase=0 elapsed=84s on=False
```

토글 OFF 후 추적 — phase 5(30초) → 0 자동 복귀

The graph displays the concentration of two gases, CO (blue line) and CO₂ (green line), in ppm over a 50-minute period. The y-axis represents concentration in ppm, ranging from 0 to 100. The x-axis represents time, with labels every 10 minutes from 10:00 to 10:50. A red horizontal line is drawn at 50 ppm, likely indicating a safety threshold. The CO concentration starts at approximately 80 ppm, peaks at 100 ppm at 10:10, and then fluctuates, with a significant drop to 10 ppm at 10:40. The CO₂ concentration remains relatively stable, fluctuating between 5 and 15 ppm throughout the period.

Time	CO (ppm)	CO ₂ (ppm)
10:00	80	10
10:05	90	10
10:10	100	10
10:15	80	10
10:20	95	10
10:25	90	10
10:30	85	10
10:35	80	10
10:40	10	10
10:45	95	10
10:50	85	10

수신측 복귀 — H₂S 0.35ppm ‘정상’ 재분류 (검③ 그림 4와 대비)



Grafana H₂S 전체 곡선 — peak 유지 후 OFF 시점 급락, baseline 복귀유지

정량 검증 — 생성 의도 대비 분류 정합률

생성기가 매 데이터에 동봉한 expected_status(의도)와 수신 서버가 실제 분류한 status를 DB에서 직접 대조하였다 (PostgresSQL psql 직접 조회, G3 시나리오 데이터 n=1,040). 결과 일치율은 95.2%였다.

expected_status	status	count	n	match_pct
caution	caution	8	1040	95.2
caution	normal	1	(1 row)	
danger	caution	10		
danger	danger	198		
normal	caution	39		
normal	normal	784		

일치율 95.2%

교차표 — expected_status × 실제 status (n=1,040)

불일치 50건(4.8%)은 전수가 설계로 설명된다.

normal→caution 39건은 phase 1(전조)의 의도된 특성이다.

라벨은 normal이지만 목표 분포(7 ± 2ppm)가 주의 임계(10ppm)에 인접해 일부 표본이 임계를 넘도록 설계되어 있다 (AI가 이 구간을 조기 탐지하면 lead-time 성과로 평가하기 위함). danger→caution 10건과 caution→normal 1건은 phase 전환 직후 OU 평균회귀가 새 목표값에 도달하기까지의 약 3틱 ramp 구간에서 발생한 경계 지연이다.

즉 분포 중심부에서의 순수 오분류는 0건이며, 임계 분류기는 생성기의 의도를 사실상 완전하게 재현하고 있다.

위험구역 자동 발동 배선 (P-AZ)

검증 중 발견한 설계 의도와 구현 사이의 공백을 auto_trigger 모듈로 보완

발견 — 위험인데 구역 미생성

센서가 위험 분포되자도 위험구역이 생성되지 않음

원인: 위험(trigger_source='threshold')는 설계 미흡

그런데 실제로 수신 종료에 발동 코드 있음

그리고 이어서 실제 의도와 구현 사이의 공백

보완 — 모듈 신설·경로 연결

geofence/auto_trigger.py 신설 → 수신 종료 연결

임계 분포 함수(원형) 변경이 수행되지 않고, 임계 상수만 인터페이스로 제공됨

O₂ 제외 (구현됨)

현재 생성기

① 책임 분리

발동은 가스_changer ‘변이’ 시점 ‘10초’ 전행.

반응 시간 ‘10초’ 증가 · 종료는 기존 로직으로 진행

② 중복 방지 이중화

Redis 원자 제어와 SET NX EX → DB 활용 가능 · DB 중복으로 충돌 발생 적다.

③ 장애 격리

오류 내역에서 오든 에러 함수 · 위험구역 차기 실행/가 데이터 수정 종료로 제한되어 있음.

재검증 결과 (검③ 그림 10)

위험 전이 시점엔 위험구역이 자동 생성됨을 확인. Django pod 3개 스케일아웃 상태에서 구역이 정확히 3개만 생성 — 다중 replica 환경에서 데이터가 의도대로 유동·발동을 지시함을 입증했다.

검증의 한계 및 후속 과제

정직한 기록을 위해 본 검증이 다루지 않은 범위를 명시한다.

첫째, 자동 발동 모듈의 logger.info 기록이 출력되지 않는 현상이 있었는데, 이는 Django LOGGING 설정 부재로 앱 로거의 INFO 레벨이 기본 목을 처리하기 때문으로 확인되었다(동작에는 영향 없음 — 발동 사실은 화면·DB로 교차 확인, 설정 보완 예정).

둘째, 검증 중 PostgreSQL 커넥션 고갈(too many clients) 오류를 1회 관측하였다 — 다중 pod와 CONN_MAX_AGE 조합의 커넥션 상한 검토가 필요하다.

셋째, 다중 Pod end-to-end 지연(p95)과 HPA 스케일아웃 곡선의 실측은 별도 부하시험(Locust) 항목으로 남긴다.

결론

위험 상황의 전체 수명주기 - (시나리오 전환 명령 → 상태기계 자동 진행 → 설계 분포에 따른 데이터 생성 → 수신측 임계 분류의 의도 대비 일치율 95.2%, 순수 오분류 0건) → 관계 화면·위험구역 반응 → 해소 시 정상 복귀)

위험 상황의 전체 수명주기가 실제 운영 구성(Kubernetes, 다중 replica)에서 설계대로 작동함을 단계별 증빙으로 확인 하였다.

아울러 검증 과정에서 설계와 구현 사이의 공백(임계 초과 시 위험구역 자동 발동 미배선)을 발견하고 직접 보완·재검증함 으로서, 본 검증 체계가 단순 확인을 넘어 결함 발견과 개선으로 이어지는 실효적 수단임을 함께 보였다.

한계 및 향후 과제

본 시스템은 부트캠프 평가·시연을 목적으로 하며, 다음 한계를 인지하고 있다. 먼저 인정함으로써 시스템의 성숙도를 정직하게 드러낸다.

단일 Redis-PostgreSQL은 단일 장애점(SPOF)이다. 데모 규모(센서 약 10대, 단일 노드)에서는 의도적 단순화이며, 확장 시 HA Redis·읽기 복제·용도별 Redis 분리가 다음 단계다.

임계값이 세 곳(운영 분류 코드, 관리 UI용 DB 모델, 클라이언트 차트용 JS)에 존재하여 동기화 책임이 남는다. DB를 단일 출처로 모으는 것이 정석이며 후속 1순위 과제다.

point_in_polygon 판정이 세 모듈에 중복 구현되어 있고, 동명의 NotificationLog 모델이 두 앱에 존재한다. 공용 유틸·모델 통합이 필요하나 마이그레이션을 동반하므로 발표 후 정리 대상이다.

목록 API에 기본 페이지네이션이 없어 데이터가 커지면 응답이 비대해질 수 있고(화면용 큰 테이블은 자체 페이지네이션을 구현함), 시연 전용 엔드포인트가 운영 라우팅에 노출되어 있어 환경 분기로 분리하는 것이 안전하다.

관측 측면에서 Alertmanager와 시스템 알람 규칙, CI/CD 파이프라인은 아직 구성되지 않았다. 배포 이미지에는 :latest 캐시 함정을 피하기 위한 시각 기반 고유 태그(redeploy.sh)가 이미 적용되어 있으며, 현재 로컬 pre-commit(린트·포맷)은 있으나 CI에서의 자동 테스트·검증과 커밋(SHA) 연동 태그가 후속 후보다.

운영 환경 검증에서 실측된 한계도 기록한다. 다중 Pod 환경에서 PostgreSQL 커넥션 고갈(too many clients)이 1회 관측되어, CONN_MAX_AGE와 DB 커넥션 상한의 조합에 대한 검토가 필요하다. 또한 Django LOGGING 설정 부재로 앱 로거의 INFO 레벨이 기본 목음 처리되어 로그 기반 추적이 제한되며, 설정 보안이 예정되어 있다. 배포·구성 측면에서는, 운영 중 kubectl로 직접 적용한 변경(실행 명령·probe 방식·공유 볼륨)이 매니페스트 파일에 반영되지 않는 구성 드리프트를 경험하고 파일을 라이브 상태와 1:1로 동기화했다 — 매니페스트를 단일 출처로 강제하는 GitOps 도입의 필요성을 확인한 사례다. 아울러 업로드 공유 볼륨의 ReadWriteOnce 접근 모드는 단일 노드(kind) 전제에서만 다중 Pod 공유가 성립하며, 다중 노드 확장 시 RWX(NFS 등)나 오브젝트 스토리지로의 전환이 필요하다.

부록

앱	책임
devices	센서 수신구(SensorDataView)·근접 그래프·센서 모델
alerts	경보 평가기·상태머신·AI 서비스·외부 알림·경보 모델
geofence	위험구역 확산·생애주기·이벤트·시나리오·구역 모델
realtime	WebSocket consumer·발행 래퍼·최신 상태 캐시
backoffice	마스터데이터·권한·감사·알림 디스패치·백업·통합 로그
accounts	커스텀 사용자·역할·인증 (leaf 앱)
dashboard	관제 UI·판정 오케스트레이터(CheckGeofenceView)·맵
workers	작업자·위치·푸시
mysite	설정·Celery 메트릭·내부 인증 미들웨어·ASGI
safety / vr_training	작업 전 체크리스트 / VR 안전 교육 (독립 부가기능)

상수	값	의미
DB_SAVE_INTERVAL_SEC	5	측정 저장 주기(초)
AI_INFERENCE_INTERVAL_SEC	5	AI 추론 주기(초)
RE_ALARM_INTERVAL_SEC	60	동일 경보 재발행 최소 간격(초)
CAUTION/ DANGER_COUNT_THRESHOLD	5	격상 확정 누적 횟수
RECOVERY_WINDOW_SEC / COUNT	10 / 7	회복 확정 창(초) / 횟수
_AI_PRED_THROTTLE_SEC	180	AI 예측 경보 throttle(초)
ARIMA WINDOW / MIN_TRAIN	500 / 60	학습 윈도우 / 최소 샘플
IsolationForest WINDOW / MIN_TRAIN	200 / 70	학습 윈도우 / 최소 샘플
확산 V_BASE / R_MAX	2.0 / 700	기준 확산 속도 / 최대 반경(px)
tier 승격 임계	1 / 3	confirmed / critical 확인 센서 수
TIER_TTL (잠정/확인/긴급)	300/1800/3600	tier별 수명(초)
zone tick 주기	30	위험구역 갱신 주기(초)
HPA (min/max/CPU)	2/5/70 %	Django 자동 확장

부록 C. 배포·검증 커맨드 (요약)

배포는 이미지 빌드 → 레지스트리 푸시 → 디플로이먼트 이미지 갱신 → 롤아웃의 순서로 자동화되어 있으며(redeploy.sh), 마이그레이션은 Django Pod 기동 시 initContainer가 자동 실행한다.

컨테이너 내부 검증은 디플로이먼트에서 Django 시스템 점검 명령을 실행하여 수행한다.

접속은 ingress를 통하며(sensa.localhost, grafana.localhost), Prometheus는 ingress를 거치지 않고 Pod IP를 직접 scrape한다.

그림	파일
그림 1. 전체 시스템 아키텍처	SenSa_아키텍처.png / .svg
그림 2. 데이터 파이프라인	SenSa_파이프라인_정정.png / .svg
그림 3. 위험구역 생애주기	SenSa_파이프라인_위험구역.png / .svg
그림 4. ERD	SenSa_ERD.mermaid / SenSa_ERD_설명.md
그림 5. 알림 디스패치	SenSa_파이프라인_알림디스패치.png / .svg
그림 6. 감사 로그	SenSa_파이프라인_감사.png / .svg
그림 7. 관측	SenSa_파이프라인_관측.png / .svg